COMPLETE PROJECT OVERVIEW | ARCHITECTURAL SPECIFICATION | MULTI-AGENT SECOPS | ZERO-DEPENDENCY

AEGIS-AGENT — Autonomous Security Operations Platform

Complete Technical Blueprint, Architecture Deep-Dive, Multi-Agent Pipeline, View-by-View Walkthrough, and Operator Execution Manual

REPOSITORY URL
github.com/KulakeerthiKK/AfterHours

RUNTIME STACK
Zero-Dependency Node.js + SQLite / PostgreSQL

AI INTEGRATION
OpenRouter AI (DeepSeek / Claude / Gemini)

SAFETY SCOPE
SEC-LOCAL-8080 (Non-Destructive Sandbox)

1. Executive Summary & The Problem Space

Modern application security faces a severe scalability bottleneck. Traditional **Static Application Security Testing (SAST)** tools generate high volumes of false positives without contextual understanding, while **Dynamic Application Security Testing (DAST)** tools perform blind network fuzzing that triggers firewall rate-limits and lacks source-code awareness. More critically, neither paradigm writes, tests, or verifies code patches—forcing engineering teams into manual triage, alert fatigue, and protracted remediation cycles that average **45+ days per vulnerability**.

AEGIS-AGENT solves this by introducing a closed-loop, autonomous multi-agent platform inspired by modern autonomous security engineering (such as XBOW), pairing high-contrast visual war room telemetry with rigorous mathematical and deterministic verification bounds:

Core Autonomous Capabilities:

- **Context Ingestion:** Ingests Git repositories and OpenAPI 3.0 specs to extract an Abstract Syntax Tree (AST) attack surface before sending any probes.
- **Empirical Proof-of-Exploit:** Executes non-destructive, controlled syntax breaker probes against sandboxed local endpoints to prove exploitability.
- **Automated Patch Synthesis:** Fix Agent uses LLMs (via OpenRouter AI) to write minimal, production-grade unified diff patches.
- **Dual-Check Regression Testing:** Auditor Agent verifies in an isolated worktree that the exploit is neutralized while authentic application behavior remains intact.

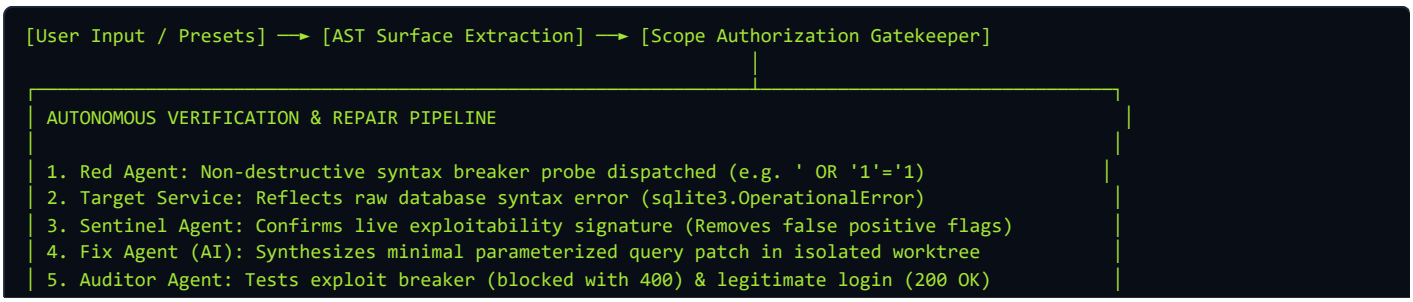
Zero-Risk Safety Governance:

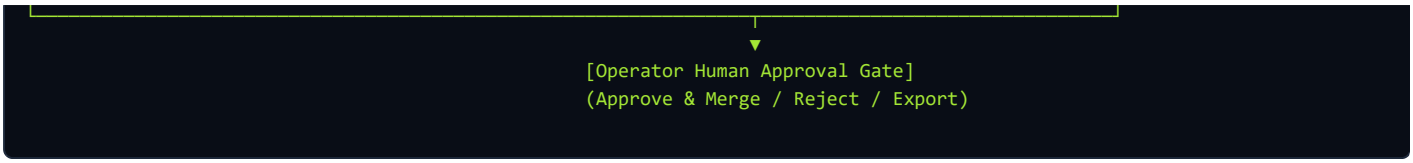
- **Strict Target Allowlisting:** Rejects all arbitrary external scanning; external URLs are safely mapped to the local container `SEC-LOCAL-8080`.
- **Human-in-the-Loop Gatekeeper:** Candidate patches are synthesized in disposable worktrees and never committed, merged, or pushed without explicit operator authorization.
- **Zero Dependencies:** Co-hosts frontend SPA and backend REST API on port `3000` using native Node.js core modules.
- **Persistent Embedded Database:** Persistent storage via native SQLite (`node:sqlite`) with optional PostgreSQL pool fallback.

2. The End-to-End Autonomous Pipeline

The workflow progresses deterministically across 6 synchronized stages from discovery to remediation:

01. Research	02. Scope Check	03. Red Agent	04. Sentinel	05. Fix Agent	06. Auditor
AST & OpenAPI Spec Ingestion	Allowlist & Sandbox Gatekeeper	Empirical Probe Execution	Proof-of-Exploit Validation	Unified Diff Patch Synthesis	Isolated Regression Verification




▼
[Operator Human Approval Gate]
(Approve & Merge / Reject / Export)

3. Detailed Architecture & Agent Roster

AGENT / MODULE	SOURCE FILE	OPERATIONAL MECHANICS	SAFETY & VERIFICATION BOUNDARY
Target Research `Stage 01`	`backend/routes.js` `js/app.js`	Parses OpenAPI specs and Git repositories, detects runtime stack (Express, Flask, Gin, Next.js), and extracts parameter attack surfaces.	Maps external staging targets (e.g. personal portfolio deployments) into local sandbox `SEC-LOCAL-8080`.
Scope Controller `Gatekeeper`	`backend/runsStore.js` `backend/redAgent.js`	Enforces allowlist policy prior to dispatching any request. Validates scope confirmation checkbox. Deduplicates rapid requests.	Blocks arbitrary public hosts; verifies `scope_confirmed === true` before state transition.
Red Agent `Stage 02`	`backend/redAgent.js`	Executes safe, predefined test cases: • `input-validation` (CWE-89 SQLi) • `authentication-check` (CWE-287 JWT) • `path-traversal` (CWE-22 Path) • `command-injection` (CWE-78 Cmd)	Zero destructive payloads. Uses safe boundary-testing strings to capture stack traces without modifying database records.
Sentinel Agent `Stage 03`	`backend/sentinelAgent.js`	Analyzes raw HTTP response codes, error payloads, and database traces to deterministically classify exploitability (`CONFIRMED` vs `NEEDS_REVIEW`).	Filters out scanner hallucinations. Halts execution if evidence is ambiguous, preventing unnecessary code changes.
Fix Agent & AI `Stage 04`	`backend/fixAgent.js` `backend/fixAgentAdapter.js`	Generates minimal unified diffs replacing string concatenations with parameterized bindings. Invokes OpenRouter LLM or deterministic fallback.	Isolated worktree synthesis. Never touches production files or branches without human sign-off.
Auditor Agent `Stage 05`	`backend/auditorAgent.js`	Executes dual-check regression matrix: 1. Security check: previous exploit payload now fails safely. 2. Functional check: legitimate authenticated queries succeed.	Deterministic independent test suite execution; records checks passed in persistent Security Memory.
Workflow Coordinator `State Machine`	`backend/coordinator.js`	Manages asynchronous stage lifecycles, timer ticks, stage retry counters, and SSE / polling state payloads.	Coordinates graceful degradation and error handling across stages.

4. User Interface Architecture & Sitemap

The application is structured as a zero-latency Single Page Application (SPA) with 8 primary views:

1. Landing Overview (`#landing`)
Split hero layout with animated typewriter terminal console (`aegis-ast-trace@local`), 4 KPI metric counters, one-click **Run Security Assessment** button, **60s Guided Tour** launcher, 6-stage interactive feature showcase, and multi-language API quick-starts (cURL, PowerShell, Node.js, Python).

2. Target Research & Ingestion (`#research`)
Input card for Git repository URL, branch ref, staging URL, and API spec. Features 4 quick preset chips (*Auth Microservice*, *Session Gateway*, *Report Exporter*, *Custom Target*). The right pane displays real-time AST surface

4. Attack Path Visualizer (`#attackpath`)
Interactive node-and-edge attack graph with animated dashed SVG connectors illustrating: `Ingress Entrypoint` → `Vulnerable Component` → `Reachable Asset Impact`. Includes slide-out Evidence Inspector Drawer for inspecting HTTP payloads.

5. Finding Inspector (`#findingdetail`)
Deep-dive vulnerability analyzer featuring CVSS 1.0–10.0 risk score badges, captured HTTP probe requests and response traces, local Semgrep AST scan triggers, SARIF JSON export, and operator Human-in-the-Loop decision controls (*Approve Patch*, *Reject Patch*, *Run Auditor Check*).

extraction, detected technology stack, and parameter risk tiers.

3. Security War Room (`#war-room`)
Mission control cockpit featuring a live multi-agent network visualizer, millisecond stopwatch (`⏱ 00:04.2s`), laser progress bar (0% → 100%), speed controls (`0.5x` , `1x` , `2x` , `Fast`), simulation state toggles (Pause/Resume, Step, Reset), and audio synthesizer chimes.

6. Governance Dashboard & Logs (`#dashboard`, `#logs`)
Searchable historical audit log of all runs with severity filters (*All*, *High*, *Medium*, *Remediated*), persistent SQLite records, and real-time streaming system telemetry logs with colored agent tags.

5. Inside the War Room: The 4-Pane Output Workbench

When a security assessment completes in the War Room, the bottom cockpit activates four detail panes:

WORKBENCH PANE	DISPLAYED DATA & EVIDENCE	OPERATOR ACTIONS & DELIVERABLES
Pane 1: Finding Dossier `Vulnerability Proof`	• Identified CWE (e.g. `CWE-89 SQL Injection`) • Affected Source Component (`authService.js:42`) • Subtabbed Evidence Viewer: HTTP Request, Response Body, Database Syntax Breaker Trace, and Subprocess Call Stack.	• **Copy cURL:** Copies ready-to-run HTTP command to replicate the probe. • **Copy Evidence:** Copies full trace logs for tickets. • **Export SARIF:** Formatted for security dashboards.
Pane 2: Proposed Fix `AI Unified Diff`	• Remediation Strategy & Technical Rationale • Syntax Compilation Check (`python -m py_compile • PASSED`) • Interactive Code Diff: Green additions (`+`) and red deletions (`-`) showing parameterized query replacement.	• **Unified vs. Split Diff:** Toggle between single-column and side-by-side views. • **Approve & Apply:** Simulates safe merge into repository. • **Reject Patch:** Rejects candidate code change.
Pane 3: Regression Audit `Independent Verification`	• Dual-Check Matrix: Security Regression (Exploit blocked) and Functional Regression (Authentic users can authenticate). • Reproducibility Banner: Displays `REMEDIATED` status with 1/1 checks passed and runtime benchmark (`38ms`).	• **Rerun Regression Test:** Triggers independent auditor re-evaluation on demand. • **Persistent Memory Record:** Updates regression tracking fingerprint in database.
Pane 4: Raw Logs `Execution Telemetry`	• Chronological system execution log with millisecond timestamps. • Multi-agent coordination state transitions (`[Scope Controller]`, `[Red Agent]`, `[Sentinel Agent]`, `[Fix Agent]`, `[Auditor Agent]`).	• Real-time inspection of HTTP status codes, headers, and isolation boundary checks.

6. Complete REST API Reference

The backend provides a RESTful API co-hosted with the frontend on port `3000`:

METHOD	ENDPOINT	PAYLOAD / PARAMETERS	RESPONSE DESCRIPTION
`GET`	`/api/health`	None	System status, database type, and AI connectivity status.
`GET`	`/api/runs`	None	List of all historical and active 5-stage workflow runs.
`POST`	`/api/runs`	`{"target_id", "test_case", "scope_confirmed", "simulation_outcome"}`	Initiates 5-stage workflow; returns `run_id` and initial state.
`GET`	`/api/runs/:id`	`:id` (Run identifier, e.g. `RUN-1050`)	Full run state, stage statuses, evidence traces, diff, and audit results.
`POST`	`/api/runs/:id/approve-fix`	`{"operator_notes": "string"}`	Records human sign-off; updates approval state in persistent database.
`GET`	`/api/runs/:id/report`	Query param: `?format=json` or `?format=md`	Generates sanitized Executive Audit Report in JSON or Markdown.
`POST`	`/api/research/ingest`	`{"repo_url", "staging_url", "branch", "api_spec_content"}`	Performs AST surface analysis, discovers endpoints, and returns risk tiers.

METHOD	ENDPOINT	PAYLOAD / PARAMETERS	RESPONSE DESCRIPTION
POST	`/api/scans`	`{"target_id": "string"}`	Triggers local AST Semgrep pattern scanner against target codebase.
GET	`/api/findings`	None	Returns all normalized vulnerabilities with CVSS scores and fingerprints.
POST	`/api/findings/:id/regression`	`:id` (Finding identifier, e.g. `FIND-CWE89-01`)	Auditor Agent re-executes independent regression test suite.
GET	`/api/security-memory`	None	Persistent regression tracking table showing detection counts and status.
POST	`/api/reset`	None	Resets in-memory workflow runs, cache, and findings to clean initial state.

7. Step-by-Step Operator Manual: How to Run & Obtain Output

Scenario A: Fastest Quick-Start (One-Click Autonomous Demo)

1. **Open the Application:** Launch your browser and navigate to `http://localhost:3000`.
2. **Click "Run Security Assessment":** On the Landing Page hero, click the lime button: `Run Security Assessment`.
3. **Watch Real-Time Execution:** The view switches automatically to the **War Room (`#war-room`)**. Watch the progress bar advance from 0% → 100% as the Scope Controller, Red Agent, Sentinel Agent, Fix Agent, and Auditor Agent coordinate synchronously.
4. **Inspect Outputs:** Once complete, inspect the captured error trace under **Finding Dossier**, review the unified diff under **Proposed Fix**, and verify that regression checks passed under **Regression Audit**.

Scenario B: Ingestion & Target Research (Custom URL / Portfolio Demo)

1. **Navigate to Target Research:** Click **Target Research** in the navbar (or visit `http://localhost:3000/#research`).
2. **Select a Preset or Custom URL:**
   • Click **Session Gateway (JWT Auth)** to test JWT algorithm "none" bypass.
   • *OR* click **+ Custom Target** and enter your own staging link (e.g. `https://abhi-portfolio-wheat-s.vercel.app`). The system automatically maps your target to the safe container `SEC-LOCAL-8080`.
3. **Ingest Target Surface:** Click the green button: **Ingest Target & Discover Surface**. The right column instantly extracts discovered endpoints, parameter risk levels (High, Medium, Critical), and technology stack.
4. **Launch Assessment:** Click **Launch Live War Room →**. The system starts the assessment with the selected target and recommended CWE test case.

Scenario C: How to Export All Output Deliverables

1. **cURL Reproduction Command:** In the War Room under the *Finding Dossier* subtab, click **Copy cURL** to copy the exact HTTP exploit command to your clipboard.
2. **Unified Git Diff (.patch):** Under the *Proposed Fix* subtab, review the additions/deletions, toggle to *Split View*, and click **Approve & Apply Patch** to simulate applying the patch.
3. **SARIF JSON Security Vulnerability File:** Navigate to **Finding Detail (`#findingdetail`)** and click **Export SARIF JSON** to download the standard SARIF vulnerability artifact for security scanners.
4. **Executive PDF Report:** Open or download the generated comprehensive PDF documentation directly from the local server: `http://localhost:3000/AEGIS_AGENT_Complete_Project_Overview.pdf`.

8. Key Differentiators & Value Proposition

EVALUATION METRIC	TRADITIONAL TOOLS (SAST / DAST)	AEGIS-AGENT AUTONOMOUS SECOPS
Triage & False Positives	High alert volume; requires manual code auditing and verification for every ticket.	**Zero False-Positive Gate:** Sentinel Agent requires empirical HTTP exploit reproduction before flagging.
Remediation Cycle Time	Industry average of 45+ days from vulnerability discovery to developer pull request.	**Under 60 Seconds:** Closed-loop pipeline synthesizes and verifies a minimal unified diff in real-time.
Patch Reliability	LLM-generated code snippets often hallucinate non-existent libraries or break existing endpoints.	**Dual Regression Testing:** Auditor Agent executes regression suites in isolated worktrees before human approval.
Deployment Friction	Heavy enterprise agents, expensive SaaS subscriptions, and complex CI/CD dependencies.	**Zero Dependencies:** Single portable codebase, runs locally via native Node.js and persistent SQLite.